

Global Aviation Recovery: Did the Skies Really Go Empty during COVID-19?

Ranjith Kumar Pachaiyappan
David Ajayi

Data Visualisation Assignment (CSC1143/CSC1175)

Abstract

Our work seeks to investigate the pervasive narrative that emerged during the early months of 2020 which we are saying is the idea that "the skies went completely empty" as the COVID-19 pandemic swept across the globe. Eventhough Commercial Airtransport is undeniable it also faced the most significant crisis in history, we sought to move beyond anecdotal evidence and quantify exactly how much global air traffic dropped. More importantly, we investigated whether this collapse was uniform across different world regions or if distinct "bubbles" of activity remained.

To address this question, we collected the massive "Crowdsourced air traffic data from the OpenSky Network (2019–2020)" dataset to further analyze that. This investigation required processing around 53 million flight records, a volume that required the use of high-performance GPU computing (RAPIDS cuDF). By aggregating this data and normalizing daily flight counts against a 2019 "prepandemic" baseline, our final visualization reveals a nuanced reality. While Europe experienced a near-total collapse dropping to approximately 10% of its normal traffic—other regions such as Asia and North America showed remarkable resilience, maintaining activity levels between 40% and 60%. This distinct divergence disproves the myth of a uniform global shutdown and highlights how domestic markets and cargo networks kept certain skies busier than others.

1 Dataset(s)

We acquired our primary data from the Zenodo scientific archive, specifically the dataset titled "*Crowd-sourced air traffic data from the OpenSky Network 2019–2020*". This dataset was originally published to support research into the pandemic's impact on climate and mobility.

Volume and Scale

The dataset serves as an excellent example of "Big Data" primarily due to its **Volume**.

- **Raw Size:** The data is distributed as 24 individual compressed CSV files (one per month), approximately 11GB of total raw text data.
- **Row Count:** Once we merged these files, the dataset contained **53,484,007 rows**. Each row represents a single flight vector OpenSky sensor network collects .
- **Attributes:** The dataset contains 16 columns, including technical details like aircraft transponder IDs. We filtered this down to the essential fields for our story: day, origin (the ICAO code of the departure airport), destination, callsign, and typecode (model of the aircraft).

Variety and Integration

The second aspect of Big Data present in our project is **Variety**. The flight data alone is geographically opaque; it provides airport codes (like "EIDW" or "KJFK") but does not contain country or continent information. To tell a story about global regions, we had to integrate a secondary dataset: `airports.csv`.

This secondary dataset is a static lookup table containing metadata for over 13,000 airports world-wide. By joining these two unique sources on a high velocity stream of sensor data, the other static geospatial database, we were able to map millions of flights to their respective regions (Europe, North America, Asia, etc.). This integration was very important, because without it, we would only have a count of flights per airport, which would be too granular to visualize effectively on a single chart.

2 Data Exploration, Processing, and Integration

The process of transforming 53 million raw flight records into a one and also clean visualization was the most technically challenging aspect of this assignment. We utilized a cloud-based environment (Google Colab) to perform this work, being standard spreadsheet software (Excel) or even local Python, they would crash mainly because of memory limitations in our PC(which is a laptop).

Technical Challenges: Memory and Speed

Our initial attempt to load the data using the standard Python `pandas` library failed immediately because the 11GB dataset exceeded the available RAM on our standard Google Colab runtime. To overcome this, we pivoted to using **RAPIDS cuDF** and **dask_cudf**. These libraries allow for GPU-accelerated dataframe manipulation. To our luckiness, Google offers a NVIDIA T4 GPU, so using that GPU, we were able to process the data in chunks, reducing processing time from hours to mere minutes.

Data Cleaning Pipeline

Once the data was ingestible, we encountered several quality issues that required cleaning:

1. **Timezone Normalization:** The `day` column in the raw data was inconsistent, with some timestamps including UTC offsets and others are not. This caused the same "day" to be split into two separate entries in our aggregation. We wrote a cleaning function to normalize all timestamps to strict UTC midnight.
2. **Filtering Invalid Flights:** The dataset includes noise, such as ground tests or incomplete records. We filtered out any rows where the `origin` was missing, as we could not assign these flights to a region.

Geospatial Integration & Logic

A simple join with the airport database was not enough. We noticed that "Asia" is too broad a category, as the pandemic started in China (East Asia) but affected India (South Asia) differently. To address this, we implemented custom logic to split the Asian continent into sub-regions ("East Asia", "South Asia", "Middle East"). This allowed us to see if the "East Asia" line dropped earlier than the others—a key part of our narrative. But later we realized that our ideology was wrong, the final graph looked too cluttered and we can barely be able to get any info from the graph, so we combined all ASIA into one ASIA.

Aggregation and Normalization

The final step was to prepare the data for plotting. Raw flight counts are misleading because they are scale-dependent; North America has roughly 40,000 daily flights, while Africa has far fewer. If we plotted raw numbers, the North American line would dwarf everything else.

To make it fair, apples to apples comparison, we calculated a "**Baseline 2019**" for each region. We defined this as the average number of daily flights in that region during the year 2019. We then created our primary metric, `pct_of_2019`:

$$\text{pct_of_2019} = \left(\frac{\text{Daily Flight Count}}{\text{Baseline 2019 Average}} \right) \times 100$$

This normalization was the key design decision in our processing phase. It transformed the data from a "count" (Magnitude) to a "percentage" (since it is a Relative Change), which is much better suited for answering our specific question about the *impact* of the pandemic relative to normal operations.

3 Visualisation

Design Process & Sketch

Our design process began with a pencil and paper sketch. We knew we wanted to show time on the X-axis and some measure of "activity(somehow that shows info about flights)" on the Y-axis.

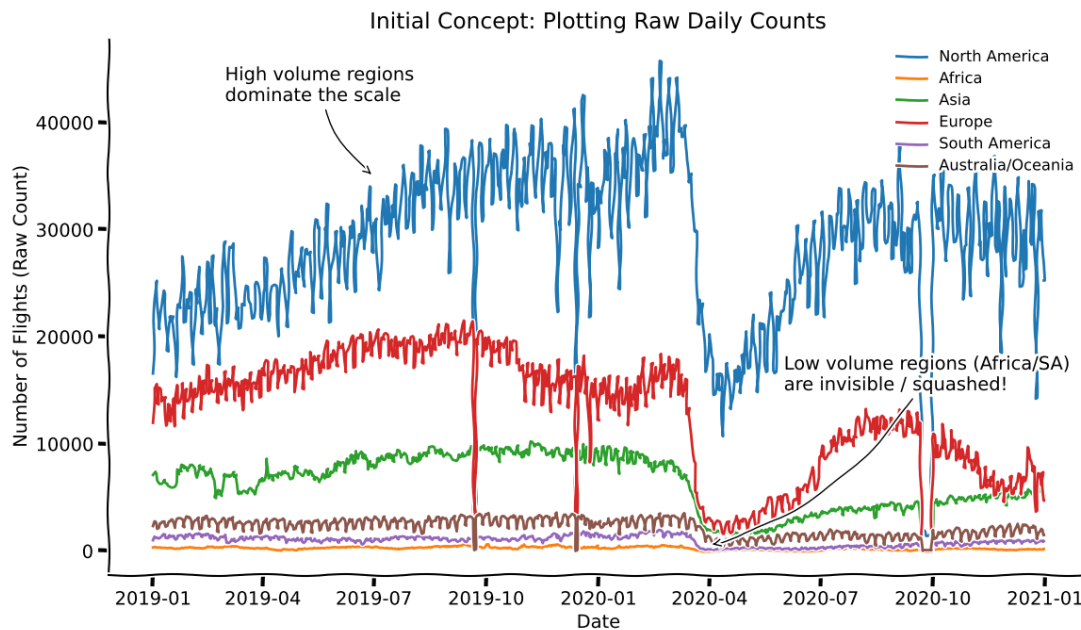


Figure 1: Initial sketch of the recovery timeline. Note that we initially planned to use raw counts, but the sketch revealed that low-traffic regions would be invisible at the bottom of the chart.

Our initial sketch highlighted a potential problem: if we used raw numbers, the lines for Africa and South America would be squashed at the bottom of the Y-axis, making their recovery trends impossible to read. This sketch directly influenced our decision (mentioned in Section 2) to switch to a percentage-based normalization.

Choice of Chart Type

We selected a **Multi-Line Time Series Chart**.

- **Why not a Bar Chart?** A bar chart would require binning the data (e.g., by month), which would hide the sudden, sharp daily drops that occurred when specific lockdowns were announced in March 2020. We needed the granularity of daily data.
- **Why not a Heatmap?** While a heatmap could show intensity, it is difficult to compare exact values across rows. We wanted the viewer to be able to see exactly when the Europe line crossed below the North America line.

The multi-line chart is the standard for continuous temporal data, allowing for immediate comparison of trends, slopes (rate of decline), and recovery levels across multiple categories.

Design Choices

We implemented the visualization using the **Altair** library in Python, which is based on the Vega Lite grammar of graphics.

Smoothing (7-Day Rolling Average)

When we first plotted the daily data, the lines were extremely jagged. This is due to the natural "seasonality" of aviation: fewer cargo and business flights fly on weekends. These jagged spikes created visual noise (cognitive load) that distracted from the overall trend. Later we and planned about this and came to a solution which is we applied a 7-day rolling average to smooth the lines. This keeps the data accurate but makes the visual flow much cleaner and easier to interpret.

Use of Color and Focus

We used a categorical color palette to distinguish the regions. However, we deliberately broke the pattern for one specific region: **Europe**. Our story focuses on how Europe was the hardest-hit region. To emphasize this:

- We made the Europe line **thicker (4px)** and fully opaque (1.0).
- We made all other regional lines thinner (1.5px) and slightly transparent (0.6).

This technique uses "pre-attentive processing" to draw the user's eye immediately to the Europe line, ensuring our viewers see the main character of our story first.

Annotations and Anchors

A chart without context is just lines on a page. We added two key types of annotations:

1. **Context Lines:** Vertical dashed lines mark the "First Lockdown" (March 2020) and the "Second Wave" (late September and early October 2020). These help the viewer correlate the data drops with real-world events.
2. **The Red Dot (Anchor):** We algorithmically calculated the lowest point reached by the Europe line and highlighted it with a large red circle. This serves as a "visual anchor." It gives the viewer a concrete number (~10%) to take away, answering the "how empty?" part of our research question definitively.

Legend Sorting

Standard legends sort alphabetically. But later we found this unhelpful. Instead, we sorted our legend based on the **final recovery value**. The region at the top of the legend (North America) is the one that is highest on the chart at the end of the timeline. This aligns the legend order with the visual order of the lines, reducing the eye movement required to match a label to a line.

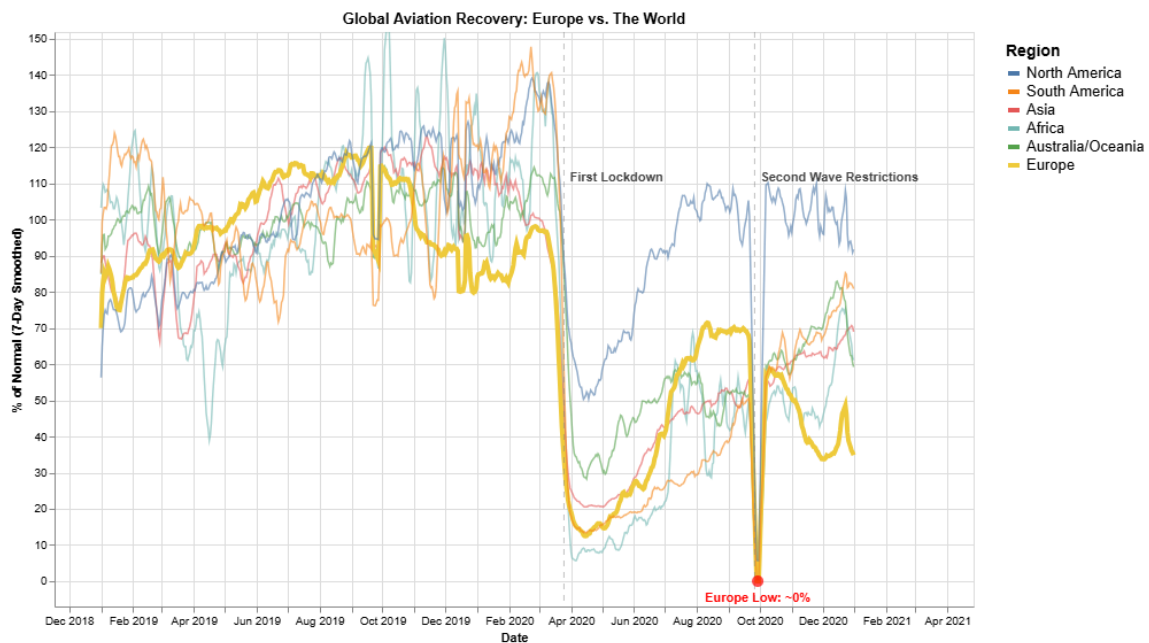


Figure 2: Final Visualisation: Global Aviation Recovery (2019–2020). The thick yellow line highlights Europe’s distinctively deep collapse compared to the resilience of North America (blue) and Asia (red).

4 Conclusion

Outcome Analysis

Our visualization successfully answers our research question: “Did the skies really go empty?” The answer is a qualified “No.” While the myth suggests a universal shutdown, the data proves that aviation activity was highly non-uniform.

The visualization clearly shows that Europe was the “sick man” of aviation during this period, dropping to very close near zero levels. This confirms that for Europeans, the skies *were* effectively empty. However, the chart reveals a surprising resilience in North America and Asia, which never fell below 10% of their 2019 levels. We hypothesize this is because the US and China(not much influence to Asia in our case here, because it was one of the most affected countries) have massive domestic aviation markets (flights within the same country) which continued a little bit even when international borders closed. Europe, being a collection of small countries, relies heavily on international travel, which was banned. Thus, our visualization hints at the underlying structural differences in global aviation markets.

Critique & Reflection

While satisfied with the result, there are aspects we would improve:

- **Domestic vs. International:** Our current chart implies the reason for the difference is domestic travel, but it doesn’t *prove* it. If we had more time, we would have liked to split the data further into “Domestic Flights” and “International Flights” to verify our hypothesis.
- **Interactivity:** While the Altair chart we generated is interactive in the HTML version (allowing zooming and tooltips), this interactivity is lost in the PDF report. A static PDF cannot convey the specific flight counts on a random day in April as well as a mouse-over tooltip can.

Collaboration

With what we are being best at, we did split our work :

- **Ranjith Kumar Pachaiyappan:** Focused on the "Backend." Did set up the Google Colab environment, managed the GPU drivers for RAPIDS, and wrote the Python code to ingest, clean, and aggregate the 53 million rows of data.
- **David Ajayi:** Focused on the "Frontend." Took the aggregated CSV and designed the visualization in Altair. Experimented with the color schemes, implemented the smoothing logic, and added the annotations to tell the story effectively. Introduced this main story idea.

We met twice in a week to discuss the "interface" between our tasks mainly, agreeing on the exact format of the CSV file that would be passed from the cleaning stage to the visualization stage. We analyzed all the data, created a visual information, and answered to our question.

References

- [1] Strohmeier, M., et al. "Crowdsourced Air Traffic Data from the OpenSky Network 2019–2020," *Earth Syst. Sci. Data*, vol. 13, pp. 357–366, 2021. Available: <https://doi.org/10.5194/essd-13-357-2021>. Accessed: <https://zenodo.org/records/5815448>.
- [2] VanderPlas, J., et al. "Altair: Interactive Statistical Visualizations for Python," *Journal of Open Source Software*, 3(32), 1057, 2018.
- [3] Evergreen, S. *Effective Data Visualization: The Right Chart for the Right Data*, SAGE Publications, 2nd Edition, 2019,
- [4] McKinney, W. "Data Structures for Statistical Computing in Python," *Proceedings of the 9th Python in Science Conference*, pp. 51-56, 2010. <https://doi.org/10.25080/Majora-92bf1922-00a>.